
name: tos-order-types description: Reference for thinkorswim (TOS) order types, entry methods, and trade management for options spreads and single-leg options on the Schwab/TOS desktop platform. Use this skill whenever the user asks about placing orders in TOS, setting up stop-losses on spreads, creating conditional orders, OCO brackets, or any order management for options trades. Also trigger when discussing closing trades, profit targets, stop-loss strategies, or any TOS platform mechanics. This covers vertical spreads, single-leg options, and stock orders. Always consult this skill before advising on TOS order entry to avoid trial-and-error in the platform. This skill is also referenced by the Options Analyzer application for programmatic order generation — Section 1 (Order Schema) and Section 2 (Decision Logic) are machine-readable specifications.

Thinkorswim (TOS) Order Types & Trade Management

Platform Context

- Broker: Charles Schwab (thinkorswim desktop)
 - Account: Self-directed IRA, Level 2 options approval
 - API note: Schwab API (formerly TDA API) uses the same order model as TOS desktop. The schemas below map to both UI entry and API submission.
-

Communication Protocol

When discussing any trade — in chat or generating automated orders — ALWAYS follow these rules:

1. **Always state the spread type using the enum name:** BEAR_PUT_DEBIT, BULL_PUT_CREDIT, BULL_CALL_DEBIT, BEAR_CALL_CREDIT. Never say “put spread” or “call spread” without the full type name.
2. **Always state the cost/credit explicitly:**
 - For debit spreads: “Your cost is \$X.XX per contract” (total: \$X.XX × quantity)
 - For credit spreads: “You receive \$X.XX per contract” (total: \$X.XX × quantity)
 - Never leave ambiguity about whether money is flowing in or out
3. **Always state what wins and what loses:**
 - “Max profit if [underlying] [rises/drops] below/above \$[strike]”
 - “Max loss if [underlying] [rises/drops] below/above \$[strike]”
4. **Example of correct trade description:**

BEAR_CALL_CREDIT on SPY 661/671 CALL You receive \$5.09 per contract (total: \$1,018.00 for 2 contracts) Max profit: \$1,018.00 if SPY stays below \$661 Max loss: \$982.00 if SPY rises above \$671 Breakeven: \$666.09

This protocol ensures clarity in conversation and maps directly to the order schema for automation.

Section 1: Order Schema

This section defines the data model for all TOS orders. Use this as the canonical reference for building order objects — whether manually in the TOS UI or programmatically via API.

1.1 Spread Types (enum: spread_type)

BEAR_PUT_DEBIT	# BUY higher strike put, SELL lower strike put. Pay debit. Prof
BULL_PUT_CREDIT	# SELL higher strike put, BUY lower strike put. Receive credit.
BULL_CALL_DEBIT	# BUY lower strike call, SELL higher strike call. Pay debit. Pr
BEAR_CALL_CREDIT	# SELL lower strike call, BUY higher strike call. Receive credi
SINGLE_LONG_PUT	# BUY put. Pay premium. Profit when underlying drops.
SINGLE_LONG_CALL	# BUY call. Pay premium. Profit when underlying rises.

```
SINGLE_SHORT_PUT # SELL put. Receive premium. Profit when underlying stays above
SINGLE_SHORT_CALL # SELL call. Receive premium. Profit when underlying stays below
```

1.2 Order Parameters

```
order:
  symbol: string          # Underlying symbol (e.g., "SPY", "QQQ")
  spread_type: spread_type # From enum above
  legs:
    - strike: number      # Strike price
      option_type: PUT | CALL
      expiration: date     # YYYY-MM-DD
      side: BUY | SELL
      quantity: integer    # Number of contracts (always positive)
      position_effect: TO_OPEN | TO_CLOSE
  price:
    type: LIMIT | STOP | STOP_LIMIT
    limit_price: number | null # Required for LIMIT and STOP_LIMIT
    stop_price: number | null  # Required for STOP and STOP_LIMIT
    stop_type: MARK | BID | ASK # For STOP orders; MARK recommended for spread
    time_in_force: DAY | GTC    # ALWAYS use GTC for protection orders
    advanced_type: SINGLE | OCO | FIRST_TRIGGERS_OCO | PAIR
    condition: null | {
      symbol: string          # Underlying symbol for condition trigger
      method: MARK | BID | ASK # MARK recommended
      trigger: GTE | LTE      # Greater/less than or equal
      threshold: number       # Price level that activates the order
    }
```

1.3 OCO Order Pair Schema

An OCO pair links two orders on the same position. When one fills, the other auto-cancels.

```
oco_pair:
  position_ref:          # The position being managed
    symbol: string
    spread_type: spread_type
    legs: [leg]          # Matches the open position
    quantity: integer
  profit_target:          # Line 1 - immediately WORKING
    order_type: LIMIT
    price: number
    time_in_force: GTC
    condition: null      # No condition - always active
  stop_loss:             # Line 2 - WAIT STOP or WAIT COND until triggered
```

```

order_type: STOP
stop_price: number
stop_type: MARK
time_in_force: GTC
condition:          # Condition on UNDERLYING price, not spread price
    symbol: string
    method: MARK
    trigger: GTE | LTE
    threshold: number

```

1.4 Spread Direction Matrix

This matrix determines order side and condition trigger direction based on spread type and intent.

spread_type	entry_side	close_side	profits_when	stop_trigger
BEAR_PUT_DEBIT	BUY	SELL	underlying DROPS	underlying RI
BULL_PUT_CREDIT	SELL	BUY	underlying RISES	underlying DR
BULL_CALL_DEBIT	BUY	SELL	underlying RISES	underlying DR
BEAR_CALL_CREDIT	SELL	BUY	underlying DROPS	underlying RI

Reading this matrix:

- `close_side` : Whether you SELL or BUY the vertical to close
- `stop_trigger_direction` : Which direction the underlying must move for you to be LOSING money, requiring a stop

Section 2: Decision Logic

Deterministic rules for building orders. A trade processing function should implement these as code.

2.1 Profit Target Calculation

```

def calculate_profit_target(spread_type, entry_price, profit_pct=0.75):
    """
    entry_price: absolute value of what was paid (debit) or received (credit)
    profit_pct: desired profit as decimal (0.75 = 75% profit)

    Returns: limit price for the closing order
    """

```

```

if spread_type in [BEAR_PUT_DEBIT, BULL_CALL_DEBIT]:
    # Debit spread: you paid entry_price, want to sell for more
    return round(entry_price * (1 + profit_pct), 2)

elif spread_type in [BULL_PUT_CREDIT, BEAR_CALL_CREDIT]:
    # Credit spread: you received entry_price, want to buy back for less
    # Keep profit_pct of the credit = buy back at (1 - profit_pct) of credit
    return round(entry_price * (1 - profit_pct), 2)

```

2.2 Stop-Loss Calculation

```

def calculate_stop_loss(spread_type, entry_price, loss_pct=0.10):
    """
    entry_price: absolute value of what was paid (debit) or received (credit)
    loss_pct: maximum acceptable loss as decimal (0.10 = 10%)

    Returns: stop/limit price for the closing order
    """
    if spread_type in [BEAR_PUT_DEBIT, BULL_CALL_DEBIT]:
        # Debit spread: spread has lost value, sell for less than paid
        return round(entry_price * (1 - loss_pct), 2)

    elif spread_type in [BULL_PUT_CREDIT, BEAR_CALL_CREDIT]:
        # Credit spread: spread has widened against you, costs more to buy back
        return round(entry_price * (1 + loss_pct), 2)

```

2.3 Conditional Trigger Price Calculation

```

def calculate_trigger_price(
    spread_type,
    underlying_price,
    current_spread_mark,
    target_spread_price, # The stop-loss price from 2.2
    long_leg_delta,      # Delta of the long leg (negative for puts)
    short_leg_delta      # Delta of the short leg (negative for puts)
):
    """
    Translates "spread reaches $X" into "underlying reaches $Y"
    using current delta to estimate the relationship.

    Returns: (trigger_price, trigger_direction)
        trigger_direction: "GTE" or "LTE"
    """
    # Spread delta = how much the spread value changes per $1 move in underlying

```

```

spread_delta = abs(long_leg_delta) - abs(short_leg_delta)

# How much does the spread need to change?
spread_change_needed = abs(current_spread_mark - target_spread_price)

# How much does the underlying need to move?
if spread_delta == 0:
    raise ValueError("Spread delta is zero - cannot calculate trigger")
underlying_move = spread_change_needed / spread_delta

# Direction depends on spread type
if spread_type in [BEAR_PUT_DEBIT, BEAR_CALL_CREDIT]:
    # These profit when underlying drops
    # Stop triggers when underlying RISES (position losing)
    trigger_price = round(underlying_price + underlying_move, 2)
    trigger_direction = "GTE"

elif spread_type in [BULL_PUT_CREDIT, BULL_CALL_DEBIT]:
    # These profit when underlying rises
    # Stop triggers when underlying DROPS (position losing)
    trigger_price = round(underlying_price - underlying_move, 2)
    trigger_direction = "LTE"

return trigger_price, trigger_direction

```

2.4 Build Complete OCO Order

```

def build_oco_closing_order(
    symbol,
    spread_type,
    legs,                # List of {strike, option_type, expiration, side, quantity}
    entry_price,         # What was paid (debit) or received (credit)
    current_spread_mark, # Current mark of the spread
    underlying_price,    # Current price of underlying
    long_leg_delta,     # Current delta of long leg
    short_leg_delta,    # Current delta of short leg
    profit_pct=0.75,
    loss_pct=0.10
):
    """
    Builds a complete OCO order pair for closing a vertical spread with
    profit target and conditional stop-loss.

    Returns: oco_pair dict matching Section 1.3 schema
    """
    profit_price = calculate_profit_target(spread_type, entry_price, profit_pct)

```

```

stop_price = calculate_stop_loss(spread_type, entry_price, loss_pct)
trigger_price, trigger_dir = calculate_trigger_price(
    spread_type, underlying_price, current_spread_mark,
    stop_price, long_leg_delta, short_leg_delta
)

# Determine close side from direction matrix
close_side = "SELL" if spread_type in [BEAR_PUT_DEBIT, BULL_CALL_DEBIT] else

# Flip leg sides for closing
closing_legs = []
for leg in legs:
    closing_legs.append({
        **leg,
        "side": "SELL" if leg["side"] == "BUY" else "BUY",
        "position_effect": "TO_CLOSE"
    })

return {
    "advanced_type": "OCO",
    "symbol": symbol,
    "profit_target": {
        "side": close_side,
        "legs": closing_legs,
        "price": {
            "type": "LIMIT",
            "limit_price": profit_price
        },
        "time_in_force": "GTC",
        "condition": None # Always active
    },
    "stop_loss": {
        "side": close_side,
        "legs": closing_legs,
        "price": {
            "type": "STOP",
            "stop_price": stop_price,
            "stop_type": "MARK"
        },
        "time_in_force": "GTC",
        "condition": {
            "symbol": symbol,
            "method": "MARK",
            "trigger": trigger_dir,
            "threshold": trigger_price
        }
    }
}

```

```
}
```

2.5 Validation Rules

These MUST be checked before submitting any order — both in UI and programmatically.

```
def validate_order(order, existing_working_orders, position):
    errors = []

    # Rule 1: No duplicate orders on same position exceeding position size
    total_closing_qty = sum(
        o.quantity for o in existing_working_orders
        if o.symbol == order.symbol and o.position_effect == "TO_CLOSE"
    )
    if total_closing_qty + order.quantity > position.quantity:
        errors.append("OVERSOLD: Total closing quantity exceeds position size. "
                      "Cancel existing orders or use OCO to link them.")

    # Rule 2: Stop-limit prices must be compatible
    if order.price.type == "STOP_LIMIT":
        if order.side == "SELL" and order.price.limit_price > order.price.stop_price:
            errors.append("CONFLICTING_PRICES: Limit price far above stop price – "
                          "order will never fill. Use OCO with separate orders instead.")

    # Rule 3: Time in force for stops
    if order.price.type in ["STOP", "STOP_LIMIT"] and order.time_in_force == "DAY":
        errors.append("DAY_STOP_WARNING: Stop order set to DAY will expire at close. "
                      "Change to GTC for persistent protection.")

    # Rule 4: Spread stops should use conditional triggers
    if (order.price.type == "STOP"
        and len(order.legs) > 1
        and order.condition is None):
        errors.append("UNRELIABLE_STOP: Stop on spread without conditional trigger. "
                      "Spread mark-based stops are unreliable in TOS. "
                      "Add condition based on underlying price.")

    # Rule 5: Condition direction matches spread type
    if order.condition:
        expected_dir = DIRECTION_MATRIX[order.spread_type]["stop_trigger_direction"]
        if order.condition.trigger != expected_dir:
            errors.append(f"WRONG_DIRECTION: Condition trigger is {order.condition.trigger} "
                          f"but {order.spread_type} stop should trigger on {expected_dir}")

    # Rule 6: Credit spread stop price must be HIGHER than credit received
    # (costs more to buy back = loss)
```



```

if (order.spread_type in [BULL_PUT_CREDIT, BEAR_CALL_CREDIT]
    and order.intent == "stop_loss"
    and order.price.limit_price <= order.entry_price):
    errors.append("CREDIT_STOP_TOO_LOW: Stop price for credit spread must be
                  "higher than credit received (you're buying back at a loss).

return errors

```

Section 3: TOS Platform Constraints

Hard limitations and behaviors of the TOS platform that affect order logic.

3.1 What Works and What Doesn't

Order Type	On Stocks	On Single Options	On Spreads	Notes
LIMIT	✓	✓	✓	Reliable
STOP (mark-based)	✓	⚠ (wide spreads)	✗	Spread mark
STOP LIMIT	✓	⚠	✗	Same mark
CONDITIONAL (underlying)	✓	✓	✓	Best trigger
OCO	✓	✓	✓	Required
OCO + CONDITIONAL	✓	✓	✓	RECOMMENDED

3.2 TOS Default Behaviors (override these)

Behavior	Default	Override To	Why
Stop order TIF on spreads	DAY	GTC	DAY
Stop type on spreads	MARK	MARK	Keep
OCO line count when created from position	1	Add 2nd	TOS

3.3 TOS Order Statuses

Status	Meaning	Action Required
WORKING	Order is live, actively seeking fill	None — monitorin
WAIT STOP	Waiting for spread mark to hit stop price	Monitor — may no
WAIT COND	Waiting for conditional trigger on underlying	None — reliable
FILLED	Order executed	Cancel linked OC
CANCELLED	Order was cancelled by user or system	Review if unexpe
REJECTED	Order rejected by TOS	Check: oversold,

Section 4: TOS UI Walkthrough (for manual order entry in chat)

Use this section when guiding manual order entry through the TOS desktop interface.

4.1 How to Access the Hidden Gear Icon (Order Rules / Conditions)

The gear icon is NOT visible by default. You must:

1. Set up your order in the Order Entry panel at the bottom of the Trade tab
2. Hover your mouse over the **far right side** of the order entry row
3. The gear icon appears on hover — click it
4. This opens the **Order Rules** window
5. The **Conditions** section is at the bottom of the Order Rules window
6. Click in the empty row under Symbol/Method/Trigger/Threshold to add a condition
7. Fill in: Symbol (underlying), Method (Mark), Trigger ($\geq$ or $\leq$), Threshold (price)
8. Click Save, then Confirm and Send

4.2 How to Enter an OCO with Conditional Stop

Prerequisites: Cancel any existing working orders on this spread first.

CRITICAL SEQUENCING — TOS will reject or overwrite orders if you don't follow this exact sequence:

1. Right-click your spread position → **Create closing order** → select the vertical close option
2. **FIRST:** Set this order's price to your profit target limit price
3. **SECOND:** Change the **Advanced Order** dropdown from "Single Order" to **OCO**
 - You MUST flag it as OCO BEFORE adding the second order line
 - If you add the second order first, it will REPLACE the first order instead of linking to it
4. **THIRD:** Now add the second vertical spread order for the same legs
5. TOS will show an error: "Orders in the same OCO group must differ at least by instrument, side, price, type or exchange"

- This is expected — changing the second order to **STP MARK** clears the error

6. **Line 1 (profit target):**

- Price: your profit target limit price
- Order type: LIMIT
- TIF: GTC
- No condition needed

7. **Line 2 (stop-loss):**

- Price: your stop price
- Order type: STOP, stop type MARK
- TIF: GTC (MUST match Line 1 — both lines must have same TIF)
- Click gear icon on THIS line only → add condition on underlying price

8. **Special Instruction:** Leave as **NONE** on both lines

9. Click **Confirm and Send**

10. Verify in Working Orders:

- Line 1 shows **WORKING**
- Line 2 shows **WAIT STOP** (with condition active)
- Both share the same OCO group number

TOS OCO Rules:

- Both orders in an OCO group MUST have the same Time in Force (DAY or GTC)
- Both orders MUST differ by at least one of: instrument, side, price, type, or exchange
- The profit target (LIMIT) and stop-loss (STOP MARK) satisfy the “differ by type” requirement

4.3 How to Close a Spread at Market (emergency exit)

1. Right-click position → Create closing order
2. Change order type to **MARKET**
3. Confirm and Send

4. **Expect slippage** — market orders on spreads fill at the natural (worst) price

5. Cancel any remaining working orders on the same spread

4.4 Advanced Order Type Selection Guide

Scenario	Advanced Order Type
Profit target + stop-loss on existing position	OCO
Entry with automatic profit/stop after fill	1st Triggers OCO
Sequential: order B activates only after order A fills	1st Triggers Seq
Two unrelated orders submitted together	Pair (use with caution)
Single order with condition	Single + gear icon condi

4.5 Special Instructions Reference

The "Special Instruction" dropdown appears in the Order Rules window. For spread orders, always leave as NONE.

Instruction	What It Does	Use For
NONE	Standard execution. Partial fills OK. Default behavior.	YES — a
FOK	Fill or Kill. Fill entire order immediately or cancel.	NO — u
AON	All or None. Only fill if all contracts available at once.	NO — u
DNR	Do Not Reduce. Prevent price adjustment on ex-div dates.	NO — un
AON/DNR	Combination of both.	NO — un

Why NONE: SPY and QQQ options are extremely liquid. Partial fills are not a concern, and the other instructions add unnecessary constraints that can prevent your orders from filling.

Section 5: Schwab API Mapping (for Options Analyzer automation)

When the Options Analyzer sends orders programmatically via the Schwab API, map the schema from Section 1 to the API format.

5.1 API Order Structure (Schwab/TDA format)

```
{  
  "orderType": "LIMIT",  
  "session": "NORMAL",
```

```
"duration": "GOOD_TILL_CANCEL",
"price": "4.95",
"orderStrategyType": "OCO",
"childOrderStrategies": [
  {
    "orderType": "LIMIT",
    "session": "NORMAL",
    "duration": "GOOD_TILL_CANCEL",
    "price": "4.95",
    "orderLegCollection": [
      {
        "instruction": "SELL_TO_CLOSE",
        "quantity": 2,
        "instrument": {
          "symbol": "SPY_041726P650",
          "assetType": "OPTION"
        }
      },
      {
        "instruction": "BUY_TO_CLOSE",
        "quantity": 2,
        "instrument": {
          "symbol": "SPY_041726P640",
          "assetType": "OPTION"
        }
      }
    ]
  },
  {
    "orderType": "STOP",
    "session": "NORMAL",
    "duration": "GOOD_TILL_CANCEL",
    "stopPrice": "2.16",
    "orderLegCollection": [
      {
        "instruction": "SELL_TO_CLOSE",
        "quantity": 2,
        "instrument": {
          "symbol": "SPY_041726P650",
          "assetType": "OPTION"
        }
      },
      {
        "instruction": "BUY_TO_CLOSE",
        "quantity": 2,
        "instrument": {
          "symbol": "SPY_041726P640",
```

```

        "assetType": "OPTION"
    }
}
]
}
]
}

```

5.2 Option Symbol Format (OCC)

Format: SYMBOL_MMDDYYC/PSTRIKE

Example: SPY_041726P650 (SPY April 17 2026 Put 650)

QQQ_041726P575 (QQQ April 17 2026 Put 575)

5.3 API Instruction Mapping

Action	API instruction
-----	-----
Buy to open a long leg	BUY_TO_OPEN
Sell to open a short leg	SELL_TO_OPEN
Buy to close a short leg	BUY_TO_CLOSE
Sell to close a long leg	SELL_TO_CLOSE

5.4 Conditional Orders via API

Important: The Schwab API has limited support for conditional orders compared to the TOS desktop UI. The desktop Order Rules / gear icon conditions (trigger on underlying price) may need to be implemented as application-level logic in the Options Analyzer rather than native API conditional orders.

Recommended approach for automation:

1. Submit the profit target as a native API OCO order
2. Implement the conditional stop as a **monitoring loop** in the application:
 - Poll the underlying price at regular intervals
 - When underlying crosses the trigger threshold, submit the stop-loss order via API
 - Cancel the profit target OCO partner if the stop fills
3. This gives you the same behavior as TOS desktop conditionals but with full programmatic control

```
# Pseudocode for automated conditional stop monitoring
async def monitor_conditional_stop(position, trigger_price, trigger_dir, stop_ord
    while position.is_open:
        current_price = await get_underlying_price(position.symbol)

        if trigger_dir == "GTE" and current_price >= trigger_price:
            await submit_order(stop_order)
            await cancel_profit_target(position.oco_partner_id)
            break
        elif trigger_dir == "LTE" and current_price <= trigger_price:
            await submit_order(stop_order)
            await cancel_profit_target(position.oco_partner_id)
            break

        await sleep(interval_seconds)
```

Appendix: Quick Reference Card

Debit Spread Closing Cheat Sheet

You PAID to enter → you SELL to close

Profit target: SELL at $\text{entry_price} \times (1 + \text{profit_pct})$ e.g., $\$2.83 \times 1.75 = \$4.$

Stop-loss: SELL at $\text{entry_price} \times (1 - \text{loss_pct})$ e.g., $\$2.83 \times 0.90 = \$2.$

Condition: Underlying $\geq$ trigger (you lose when it rises)

Credit Spread Closing Cheat Sheet

You RECEIVED to enter → you BUY to close

Profit target: BUY at $\text{entry_credit} \times (1 - \text{profit_pct})$ e.g., $\$4.70 \times 0.25 = \$1.$

Stop-loss: BUY at $\text{entry_credit} \times (1 + \text{loss_pct})$ e.g., $\$4.70 \times 1.10 = \$5.$

Condition: Underlying $\leq$ trigger (you lose when it drops)

Delta Trigger Calculation Cheat Sheet

$\text{spread_delta} = \text{abs}(\text{long_leg_delta}) - \text{abs}(\text{short_leg_delta})$

$\text{spread_loss_needed} = \text{current_mark} \times \text{loss_pct}$

$\text{underlying_move} = \text{spread_loss_needed} / \text{spread_delta}$

$\text{trigger} = \text{underlying_price} + \text{underlying_move}$ (for bear debit / bear credit spread)

$\text{trigger} = \text{underlying_price} - \text{underlying_move}$ (for bull credit / bull debit spread)

Caveat: Deltas shift daily. Recalculate weekly.

